

Module 8

Advanced Python Programming

1. Printing on Screen

Definition: The `print()` function in Python is used to display output on the screen. It can also format output using f-strings or the `format()` method.

Theory: `print()` displays messages or variables on the console. Formatted strings allow inserting variable values inside text using `f"{}"` or `.format()`.

Lab Program: Print a formatted string using `print()` and f-string.

```
name = "Arbaz"
age = 20
print(f"My name is {name} and I am {age} years old")
```

Practical Example 1: Print “Hello, World!”.

```
print("Hello, World!")
```

2. Reading Data from Keyboard

Definition: The `input()` function reads data entered by the user from the keyboard.

Theory: Data entered using `input()` is stored as a string by default. It can be converted to other data types using `int()`, `float()`, etc.

Lab Program: Read name and age from the user and print formatted output.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Name: {name}, Age: {age}")
```

Practical Example 2: Read string, integer, and float from keyboard.

```
text = input("Enter a string: ")
num = int(input("Enter an integer: "))
decimal = float(input("Enter a float: "))
print(text, num, decimal)
```

3. Opening and Closing Files

Definition: Files are used to store data permanently. Python provides the `open()` function to open files and `close()` to close them.

Theory: Files can be opened in different modes such as `r` (read), `w` (write), `a` (append), `r+`, and `w+`.

Lab Program: Open a file in write mode and write text.

```
file = open("sample.txt", "w")
file.write("This is a sample file.")
file.close()
```

Practical Example 3: Create a file and write a string into it.

```
f = open("data.txt", "w")
f.write("Hello Python File")
f.close()
```

4. Reading and Writing Files

Definition: File operations allow reading and writing data in files using methods like `read()`, `readline()`, and `write()`.

Theory: `read()` reads entire file content, `readline()` reads one line, `readlines()` reads all lines. Writing is done using `write()` or `writelines()`.

Lab Program 1: Read file contents and print them.

```
f = open("data.txt", "r")
print(f.read())
f.close()
```

Lab Program 2: Write multiple strings into a file.

```
f = open("data.txt", "w")
f.writelines(["Line1\n", "Line2\n", "Line3\n"])
f.close()
```

Practical Example 4: Create file and write string.

```
f = open("test.txt", "w")
f.write("Python File Handling")
f.close()
```

Practical Example 5: Read file and print data.

```
f = open("test.txt", "r")
print(f.read())
f.close()
```

Practical Example 6: Check file cursor position using `tell()`.

```
f = open("test.txt", "r")
print(f.tell())
f.read(5)
print(f.tell())
f.close()
```

5. Exception Handling

Definition: Exception handling is used to handle runtime errors using `try`, `except`, and `finally`.

Theory: try block contains risky code, except handles errors, and finally always executes.

Lab Program 1: Handle division by zero.

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))
    print(a/b)
except ZeroDivisionError:
    print("Division by zero not allowed")
```

Lab Program 2: Demonstrate multiple exceptions.

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))
    print(a/b)
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Practical Example 7: Handle calculator exception.

```
try:
    x = int(input())
    y = int(input())
    print(x/y)
except:
    print("Error occurred")
```

Practical Example 8: Handle multiple exceptions.

```
try:
    f = open("file.txt")
    print(10/0)
except FileNotFoundError:
    print("File not found")
except ZeroDivisionError:
    print("Division error")
```

Practical Example 9: Use finally block.

```
try:
    f = open("data.txt")
finally:
    print("File operation attempted")
```

Practical Example 10: Custom exception.

```
class MyError(Exception):
    pass
```

```
raise MyError("Custom Exception")
```

6. Class and Object (OOP Concepts)

Definition: A class is a blueprint for creating objects. Objects contain attributes and methods.

Theory: Classes define properties and behaviors. Variables inside classes can be local or global.

Lab Program: Create a class and access properties.

```
class Student:
    name = "Arbaz"

s = Student()
print(s.name)
```

Practical Example 11: Access class properties using object.

```
class Person:
    name = "Ali"
    age = 22

p = Person()
print(p.name, p.age)
```

Practical Example 12: Local and global variables.

```
x = 10
class Test:
    x = 20
    def show(self):
        print("Local:",self.x)
        print("Global:",x)

t = Test()
t.show()
```

7. Inheritance

Definition: Inheritance allows one class to inherit properties and methods from another class.

Theory: Types include single, multilevel, multiple, hierarchical, and hybrid inheritance.

Practical Example 13: Single inheritance.

```
class A:
    def show(self):
        print("Class A")

class B(A):
    pass

b = B()
b.show()
```

Practical Example 14: Multilevel inheritance.

```
class A:
    def show(self):
        print("A")

class B(A):
    pass

class C(B):
    pass

c = C()
c.show()
```

Practical Example 15: Multiple inheritance.

```
class A:
    def a(self):
        print("A")

class B:
    def b(self):
        print("B")

class C(A,B):
    pass

c = C()
c.a()
c.b()
```

Practical Example 16: Hierarchical inheritance.

```
class A:
    def show(self):
        print("A")

class B(A):
    pass

class C(A):
    pass
```

Practical Example 17: Hybrid inheritance.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B,C): pass
```

Practical Example 18: Using super().

```
class A:
```

```

def show(self):
    print("Parent")

class B(A):
    def show(self):
        super().show()
        print("Child")

b = B()
b.show()

```

8. Method Overloading and Overriding

Definition: Method overloading allows methods with the same name but different parameters. Method overriding redefines a parent method in the child class.

Practical Example 19: Method overloading.

```

class Demo:
    def add(self,a,b,c=0):
        print(a+b+c)

d = Demo()
d.add(2,3)
d.add(2,3,4)

```

Practical Example 20: Method overriding.

```

class A:
    def show(self):
        print("Parent Method")

class B(A):
    def show(self):
        print("Child Method")

b = B()
b.show()

```

9. SQLite3 and PyMySQL (Database Connectors)

Definition: SQLite3 and PyMySQL allow Python programs to connect with databases and execute SQL queries.

Lab Program: Connect to SQLite3 database and create table.

```

import sqlite3
conn = sqlite3.connect("student.db")
cur = conn.cursor()
cur.execute("CREATE TABLE student(id INTEGER, name TEXT)")
conn.close()

```

Practical Example 21: Create database and table.

```

import sqlite3

```

```

conn = sqlite3.connect("test.db")
c = conn.cursor()
c.execute("CREATE TABLE data(id INTEGER,name TEXT)")
conn.commit()
conn.close()

```

Practical Example 22: Insert and fetch data.

```

import sqlite3
conn = sqlite3.connect("test.db")
c = conn.cursor()
c.execute("INSERT INTO data VALUES(1,'Arbaz')")
conn.commit()

for row in c.execute("SELECT * FROM data"):
    print(row)
conn.close()

```

10. Search and Match Functions

Definition: Python's re module provides functions for pattern matching. re.search() searches a pattern anywhere in the string, while re.match() checks only at the beginning.

Lab Program 1: Search word using re.search().

```

import re
text = "Hello Python"
result = re.search("Python", text)
print(result)

```

Lab Program 2: Match word using re.match().

```

import re
text = "Hello Python"
result = re.match("Hello", text)
print(result)

```

Practical Example 23: Search word in string.

```

import re
print(re.search("cat","The cat is sleeping"))

```

Practical Example 24: Match word in string.

```

import re
print(re.match("The","The cat is sleeping"))

```